

Document number: P2466R0
Date: 2021-10-13
Audience: SG21
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

The notes on contract annotations

This document describes the author's vision and design goals for contract support in C++. This vision is implemented in [P2388]. (It should be noted that [P2388] has been designed minimally so that it implements more than one vision for contracts.) The goal is to build a common understanding of the goals and limits of contract support in C++, and to highlight some implementation difficulties and design constraints.

The function contract

The *function contract* is all the information that comes with the function that enables the programmer to use it correctly and in accordance with the author's intentions. Given the following example:

```
bool is_in_range(int val, int lo, int hi)
    // checks if val is in closed range [lo, hi]
;
```

Here, the function contract is communicated by a number of means:

1. The number and types of function parameters: you know that you cannot pass a string to this function, and this will be enforced by the compiler.
2. The name of the function, and the names of function parameters: you now know that you will be using this function to check if an `int` value falls between two other `int` values; you also know that the first argument will represent the to-be-tested value, and the second and the third argument represent the bounds of the range of acceptable values.
3. The comment: in case the above information isn't enough, the comment can provide the missing information, for instance that `lo` and `hi` represent a *closed* range.

Every function has a contract. Sometimes it is poorly communicated, or not communicated at all. For instance, imagine that the same function was declared with a different name, different parameter names and without a comment:

```
bool f(int a, int b, int c);
```

We would not know when and how to use it correctly.

Programmers have to know the function contract in order to use the function. However, the function contract cannot be reflected in the language. Parts of it can, but not all of it. For instance, consider this function:

```
bool all_values_match(std::vector<int> const& values, std::function<bool(int)> criterion)
    // Given the vector of positive integer values,
    // the function checks if all the values match the
    // criterion represented by the function object.
    //
    // Precondition:
    // * Every value stored in the vector is positive.
    // * Criterion isn't a nullptr.
    // * Function inside criterion can be invoked for positive values
    //   (it doesn't have to work for negatives or zero).
;
```

Parts of this function contract can be statically enforced by the compiler. For instance, if the first argument to the function is `std::vector<int>` or convertible. For different parts of the contracts, we could perform a runtime test: inspect each value in the vector, or check if `criterion` isn't `nullptr`. However, for the last part of the precondition above, there is nothing a machine can do in C++. Lots of function contracts contain this kind of not enforceable property. Therefore, the approach to "handling" the function contract in general has always been, and will remain this:

The users of the function familiarize themselves with the contract, and then it is their responsibility to fulfill their part of the contract: the precondition.

Function contract vs interface

Is function contract part of the function's interface? The answer to this question may be yes or no, depending on what your definition of an "interface" is.

If the "interface" is the type of the function: anything that the type system can distinguish, then the answer is no. However, if you define "interface" as everything you have to know to be able to use the function correctly, then the answer is: the interface is the same thing as the function contract.

Contract annotations

Contract annotations are a language feature, we try to add to C++. Their goal is to **express** a **subset** of a function contract.

“A subset”

The goal for the feature "contract annotations" is well established: we want to allow the programmers to communicate in a formal way these parts of the contract that can be expressed as *predicates* evaluated either upon an entry to a function (a precondition) or upon a successful (non-exceptional) exit from a function (a postcondition).

By "predicate" we mean something more than a C++ function taking any number of arguments and returning a `bool`. This also has to meet the criteria in a mathematical sense, or even human common sense:

1. It cannot modify the state of the program.
2. It has to be easily understood by machines and humans alike as describing the properties of function inputs (or even other parts of the program).

The second part isn't formal, but we can give an illustration:

```
void fun(int a, int b, int c)
// precondition:
// * the following must evaluate to true:
//    [&] {
//        int i = x(a), j = y(a, b, c);
//        while (q(p(a, b), r(a))) {
//            j = w(a) + ++i;
//        }
//        return i > f(j) ? i - 1 : p(a, b);
//    }(a, b, c)
;
```

While the above may satisfy the first criterion for a predicate (no side effects), it does not satisfy the second. True, the program can evaluate it; maybe even some super-clever static analyzer can draw some conclusion from it, but humans reading the function declaration cannot easily tell what is their responsibility. However, if we introduce a new function in the library interface:

```
bool not_in_constellation(int a, int b, int c);
```

With the same implementation as the lambda above, assuming that "constellation" is a term understood in the context of the library's problem domain; if we use this function instead as a precondition:

```
void fun(int a, int b, int c)
// precondition:
// * not_in_constellation(a, b, c)
;
```

Now we satisfy the second criterion, even though the predicate, if evaluated, is instruction-for-instruction identical as in the previous solution.

Because the predicate (or a conjunction of predicates) cannot ever express the full function contract, not even the precondition, the meaning of the value returned from the predicate is the following

- `false` — we are sure that the function is called out of contract,
- `true` — we do not know if the function is called out of contract or not.

“To express”

Ultimately, contract annotations do not change the basic rule of the game: the programmer is still responsible for (1) learning the function contract and (2) adhering to it. We are only adding a tool that assists the programmer in their responsibility.

This assistance comes in a number of ways. First, the learning of the contract is easier because the predicate (or a portion thereof) is expressed in a more structured way. Formal descriptions provide more unambiguous information in a concise way. Second, different tools at different stages of development cycle can provide hints to the programmer:

- IDEs — when writing the code,
- static analyzers — when compiling the code, or during static analysis,
- sanitizers — for providing instrumented builds.

The primary objective of contract annotations is *not* to affect the generation of the program in the binary form, although this may be one of the outcomes.

The primary objective is for the programmers who design function contracts to be able to **communicate** in a formal way what constitutes the violation of the contract, and therefore a bug in the program.

Not all contract violations can be communicated in this way. It is not our ambition to identify all function contracts in this way. Instead, our goal is to provide a self-consistent, easy to learn, simple tool, good at detecting one type of function contract violations.

Communication of bugs doesn't yet mean the detection or prevention of bugs: these things come later as the possible ways of consuming the information provided by the programmer.

In this sense, contract annotations can be described as "ignorable". They do not relieve the programmer of the responsibility to understand and adhere to the function contract. They do not necessarily affect the resulting binary. They wouldn't have to (although they do in the current proposals) prevent the compilation for ill-formed predicates. They can be thought of as structured comments.

Contract annotations and non-reference function parameters

Contract annotations have to be something that both parties — the caller and the implementer — must be able to understand. This is why they belong in function declarations. However, non-reference function parameters complicate this picture, because they do not fully belong to the function's interface. They represent objects that are initialized before the function is called. But the function can modify the values of these objects at will, and it is only this function that can observe the values of this objects at different stages of the function execution. Reading the values of these objects in precondition and postcondition checks is tricky.

A precondition and a postcondition is something that a caller should be able to understand and make use of. So what does the caller see?

```
int min = 1;
int max = 10;

int r = select_from_range(min, max);

assert(min <= r && r <= max);
```

Can we express the expectation reflected by the `assert()` above as a contract annotation on function `select_from_range()`? Strictly speaking, we cannot because in the declaration of the function we cannot see objects `min` and `max`. We can see different objects:

```
int select_from_range(int lo, int hi);
```

However, we know that `lo` will be initialized with the value of `min`, and `hi` will be initialized with the value of `max`. We can hope that the value that `lo` has immediately after the initialization is the value that `min` had immediately before the initialization. This hope is satisfied for type `int` and in fact, it is satisfied by many types that behave in a "regular" way. In C++20 we have a concept that reflects that: `std::regular`. (Strictly speaking the concept that we are after here is `std::copy_constructible`.) However, it is not the case in general; thus, if we have a type with unusual copy semantics, our hope is gone: a mechanical runtime check on `lo` and `hi` will not reflect what the caller expects to be checked. However, for regular types we can safely express the precondition and be confident that it reflects what the caller sees:

```
int select_from_range(int lo, int hi)
    // precondition: lo <= hi
    ;
```

This works, because we have a guarantee that a runtime-precondition check is executed immediately after the function parameters are initialized: before anything in the function body, before the constructor initialization list, before the *function-try-block*. Thus, the approximation of the precondition, by using function parameters instead of function arguments, is reasonably safe.

The situation is worse for the postcondition check, because it is executed far later (just after the value is returned and the destructors of local objects executed, but before the destructors of function parameters are executed). By that time the function may have mutated its parameters multiple times. The mutation of function parameters is relatively common, so this situation is likely to occur. The result will be that in the call like this:

```
int min = 1;
int max = 10;

int r = select_from_range(min, max);

assert(min <= r && r <= max);
```

The failed postcondition check may cause the program to abort even though the assertion expressed in the caller would pass. Or vice versa: the postcondition check may pass, even though the assertion in the caller fails. Some cases where function parameters are mutated are difficult to spot in the source code:

```
string forward(string str)
{
    // ...
    return str; // implicit move
}               // postcondition would read the moved-from state
```

This problem does not apply to reference parameters, because references do not introduce new objects: only a new way to already existing objects for the caller's scope. The problem also does not occur, for `const` parameters, because then there is no question of mutating the function parameters. The remaining case (non-`const`, non-reference parameter referenced in a postcondition) can be detected statically. Such static detection does not include the cases where a function parameter is actually not mutated. Once, such situation is detected, it can be responded to in a number of ways, but none of them is devoid of problems.

One option is to just make such situation ill-formed. In order to cope with it, a programmer would have to either add `const` or drop the postcondition annotation.

Another one is to make it UB. This would work better for the cases that actually don't modify the parameter, but for other cases may give surprising results. This is what language D implements.

Another option is to employ the solution we already discussed in the case of preconditions: make another copy of such argument upon function entry, inaccessible to function body, and then inspect this copy when checking the postcondition. However, now we are really talking about making a copy. In case of initializing the function parameter it was not necessarily so:

```
vector<val> filter(vector<int> vec) noexcept
{
    // ...
    return vec; // implicit move
}

int main()
{
    std::vector<int> r = filer({1, 2, 3});
}
```

There is no copy or move involved when initializing the function parameter, and only a move when initializing the variable `r`. We would be adding a copy (with its time complexity), which was not previously there. For move-only types, this would not even compile:

```
unique_ptr<val> filter(unique_ptr<int> val) noexcept
{
    // ...
    return val; // implicit move
}
```

For copyable types, this might change the runtime complexity of the function. This may change the exception safety guarantees of the function: the newly added copy constructor may be the only potentially throwing operation in the implementation. In case of a constrained template, the way the function uses the type may go beyond what the concept requires. This may end up in either compile-time failure, or function calling operations not from the interface that the concept describes, or worse, we'll end up in the situation where the usage is within the syntactic requirements of the concept but the semantics of the copy constructor do not meet the semantic requirements of the concept.

Plus, for types that are not regular, the solution with additional copy may not solve the problem. Consider the following, not infrequent class design, where the author abuses `shared_ptr` in the name of incorrectly understood "memory safety":

```
class Book
{
    shared_ptr<string> _title = make_shared<string>(); // poor choice

public:
    string const& title() const { return *_title; }
    void set_title(string t) { *_title = std::move(t); }
    friend auto operator <=>(Book const& a, Book const& b) {
        return a->_title <=> b->_title;
    };
};
```

This class satisfies the syntactic requirements of concept `std::regular` but doesn't model the concept because it does not have the property of "value separation". If you modify the copy of the object, this also indirectly modifies the original. In the case of such type, making a copy will not help fix the postcondition. So, the solution with a copy looks more like a patch that covers 95% of the cases, and causes surprises in the remaining 5%. (the proportions are not accurate.)

This analysis suggests that such copying cannot be done silently by the compiler: the programmer needs to confirm it one way or the other. One way is to give the programmer an "imperative" way of instructing compiler: "do a copy". Another one is a more declarative approach, where a mark from the user indicates that a copy is allowed, but not required. In this case the compiler may analyze a function with a postcondition-referenced non-`const`, non-reference parameter, and if it observes no modification, it doesn't make the copy, but inspects the original instead in the postcondition. This saves the trouble of making copies, if they are not really necessary.

This subject is still to be explored.

Pointers

Note that pointers (both raw and smart) are treated as any other values in this analysis. This is because pointers model concept `std::regular`. Their value is the address: not the value of the object that we could access through this address. Compare the usage of a pointer with the usage of an `int` as an index in a global array:

<pre>void f(int * const p) // postcondition: *p == 24 { *p = 12; }</pre>	<pre>void f(int const i) // postcondition: global_array[i] == 24 { global_array[i] = 12; }</pre>
--	--

Whatever concerns are brought up for pointers, they apply equally to any type (like `int`) that can be used — directly or indirectly — to obtain access to another object, residing at any memory location. This is also the case for types like `std::span`.

The benefits of contract annotations

New information

Static analyzers can often infer function preconditions and report their violations, without any special annotations from the programmer. Consider:

```
void f(int n, int d)
{
    int v = n / d;
    // ...
}
```

Division by zero for type `int` is Undefined Behavior (UB). UB always indicates a programmer bug. If the static analyzer could see at the same time the function body, and the body of the callers (which requires a cross-translation-unit analysis), and if any of the callers passes value zero as second argument to `f`, it can safely report a potential bug.

However, even though the bug is really there, it is not necessarily a precondition violation, and the static analyzer cannot tell the difference. The contract of the function may be that the value zero for the second argument is fine, but the function author has a bug

inside an implementation: there should have been a branch at the beginning. Therefore the message from the static analyzer is poorer quality than what it could have been if the programmer explicitly stated their precondition.

Second, there is a case like this one:

```
bool is_in_range(int val, int lo, int hi)
    // checks if val is in closed range [lo, hi]
    // precondition: lo <= hi
{
    return lo <= val && val <= hi;
}
```

And we may have an obvious bug in the caller, resulting from passing the arguments in different order:

```
return is_in_range(lower, upper, value);
```

Here the static analyzer cannot detect a bug, because there is no UB in the implementation to base the analysis on. The presence of a precondition explicitly declared by the programmer, would be this additional piece of information, comparable to UB.

Also, because this contract information is in the function interface, the static analysis becomes simpler and less resource consuming, because it now involves a single translation unit.

Abort

When a program crashes when we least expect it, we are disappointed. Calling `std::abort()` causes a crash. Therefore, it is sometimes difficult to appreciate that calling `std::abort()` can be seen as a safety feature. When the program enters a state unanticipated by the programmer (this is a bug), technically there is an unbounded amount of damage that it can make across an unbounded period of time. If we can detect this situation in a running program, and abort it, this may cause damage, but the extent of the damage now has an upper bound: once the program is no longer running, it cannot continue doing damage. The only damage is that the program is no longer doing what we need it to do. However, `std::abort()` sends a signal to environment, so if there is a different process there, monitoring our program, it can take the necessary action.

Of course, there are more benefits to be had from contract annotations. This paper highlights only those that the author believes to be not obvious. Once we have an information on what constitutes a bug in the program, it can be used in many, many ways.